Visitor Pattern File System

Introduction and design

This project models files and directories in memory and runs three operations over the same tree. The Visitor pattern keeps file-system data in the node classes and places size, search, and statistics work in separate visitor classes.

The Element interface gives both node types a shared API. FileNode and DirectoryNode each implement accept(Visitor), which calls the matching overloaded visit method. This is the double-dispatch step used by each operation.

Operations

Visitor	Result
TotalSizeVisitor	Adds each file size once; directories add no bytes.
SearchVisitor	Matches a filename rule and inclusive minimum size. Paths are root-relative and sorted.
StatisticsVisitor	Counts files and directories and reports maximum depth. The chosen root has depth 0.

TreeWalker owns recursion. It visits each reachable node once, including nested and empty directories. The visitors process nodes without traversing children themselves. DirectoryNode protects the tree through addChild and returns an unmodifiable copy from getChildren.

Clean Code and extension choices

1. **Meaningful names.** Names such as minimumSizeBytes, relativePath, and maxDepth describe what a value represents; the byte suffix also states the size unit.
2. **Single Responsibility.** TotalSizeVisitor calculates bytes, SearchVisitor finds files, and StatisticsVisitor counts nodes and depth. Each class has one operation to explain.
3. **Small methods.** Client.createFileSystem builds the sample tree. StatisticsVisitor.updateMaxDepth calculates a node depth. Both methods have one clear job.
4. **Encapsulation.** DirectoryNode.addChild preserves parent and sibling rules. getChildren returns a copy that callers cannot modify.
5. **No type-checking chains.** FileNode.accept and DirectoryNode.accept dispatch to different Visitor overloads. The code does not select behavior with instanceof or reflection.

Traversal and extension trade-offs

Keeping traversal in TreeWalker gives all operations the same visit order and avoids duplicating recursion in each visitor. A new operation can be added as another visitor without changing the node classes or their accept methods. Adding a new node type requires a new visit overload and a corresponding method in every visitor, so the pattern makes new operations easier to add than new element types.

Demonstration and verification

Client builds a fixed sample tree and runs all three visitors. The same tree and filters produce the following output on every run:

```
Total size: 4648 bytes
Java files with at least 1000 bytes:
- src/Main.java
Files: 5
Directories: 3
Maximum depth: 2
```

Test summary

The Maven build ran 24 tests with no failures, errors, or skips. The test classes are ElementDispatchTest (2), TreeWalkerTest (1), TotalSizeVisitorTest (3), SearchVisitorTest (6), StatisticsVisitorTest (4), and TreeIntegrityTest (8).

Conclusion

The visitors add independent operations to one in-memory file tree. TreeWalker controls traversal, and the concrete elements select the correct visitor method through accept.

GitHub repository: https://github.com/iuve1n/SDP_Assignment_6

UML class diagram

Visitor pattern file system

